

UAV Propulsion Digital Twin — Technical Overview

A reference for what we have built and what comes next

Asick Jahir · with Claude (Anthropic)

August 2026 · updated September 2026 for Phase 3c

Part 1 — The concept

1.1 What a digital twin actually is

A digital twin is not a 3D model, not a simulator, and not a dashboard. All three of those are pieces of one. The working definition used across ISO 23247 and the serious aerospace / automotive literature is that a twin must have three ingredients present simultaneously:

1. A **virtual representation** of a specific physical asset. Not a generic “quadrotor” — *this particular airframe with serial number 001, whose motor 3 has 47 flight hours on it.*
2. A **live data link** so that when the physical asset changes, the virtual one changes too. Telemetry over MQTT, MAVLink, a maintenance-log update — anything that keeps the virtual mirror synchronized with the real thing.
3. A **feedback loop** so that the twin produces something operationally useful — an alert, a prediction, a recommended action — and that output finds its way back to a decision the operator or the vehicle acts on.

If only (1) is present you have CAD. (1) + (2) is a live dashboard. Only (1) + (2) + (3) is a digital twin. This is why the term is so abused: most products marketed as “digital twins” stop at (2). We are building all three.

1.2 Levels of ambition

Twins are stratified by what the feedback loop actually does:

Level	What it does	Example for our UAV
Descriptive	Mirrors the current state	Live cell voltages on a screen
Diagnostic	Explains anomalies against a baseline	“Motor 2 temperature is 12 °C above its expected value at this throttle”
Predictive	Forecasts future state	“Battery reaches EoL in 47 flight hours”
Prescriptive	Recommends action	“Replace motor 2 bearing before next flight”

Level	What it does	Example for our UAV
Autonomous	Executes action closed-loop	Twin reroutes flight plan to avoid stressing motor 2

Our v1 targets diagnostic + predictive. Prescriptive is where the paying-customer value sits and where we go in Phase 3. Autonomous is aspirational.

Status as of Phase 3c: diagnostic and predictive are both live. The XGBoost bearing classifier and the Gaussian-process battery RUL predictor emit their outputs on MQTT alongside the FMU physics, and the dashboard is ready to render them.

1.3 Why UAV propulsion is the right first target

We closed this in Phase 0 but worth repeating: we picked UAV / eVTOL propulsion (specifically, small multirotor electric) because it maximises leverage across our constraints. Open datasets exist for it (ALFA, NASA PCoE Li-ion, PX4 flight logs). Open simulators exist for it (ArduPilot SITL, Gazebo). Aerospace regulation does not gate research-scale work on it the way it gates manned aviation. There is a real underserved market — small commercial UAV operators and drone-service MRO shops genuinely need per-flight battery/motor health but no incumbent serves them. And the physics (BLDC motors, Li-ion batteries, propeller aerodynamics) is close enough to your aerospace propulsion background that the domain moat is real. The alternative asset choices — automobiles, manned aircraft, jet engines — each had a fatal obstacle for a solo build.

Part 2 — The roadmap

2.1 All phases at a glance

Think tank	→	Test env	→	Requirements	→	Data & tools	→	Twin dev	→	Ship
Phase 0 (done)		Phase 1 (done)		Phase 2 (done) (as Google form, synthetic responses)		Phase 3 (done through 3c – this doc)		Phase 4 (real UAV telemetry)		Phase 5 (commercial app)

Phase-by-phase status (September 2026)

- **Phase 0 — Think tank (done)** — locked scope (UAV multirotor electric), tooling (100% FOSS, Apache-2.0), six framing decisions resolved.
- **Phase 1a — Synthetic simulator + analytical twin (done)** — Docker-Compose stack with mosquitto + InfluxDB + Python simulator + Python worker + browser dashboard.
- **Phase 1b — ArduPilot SITL (done)** — real MAVLink telemetry over TCP; mavlink-bridge translates to the same JSON schema; force-arm + mission-upload plumbing worked out.
- **Phase 1b.5 — SITL parameter polish (done)** — defaults.parm with BATT_MONITOR=4, SIM_ESC_TELEM=1, mAh-failsafes disabled for continuous test loops.
- **Phase 1c — Real physics via OpenModelica FMU (done)** — Modelica model UAVPropulsion.mo covering battery equivalent circuit, per-motor thermal, per-motor

bearing damage. Compiled to .fmu via OpenModelica; consumed by fmu_runner.py with a Python fallback if the FMU won't load.

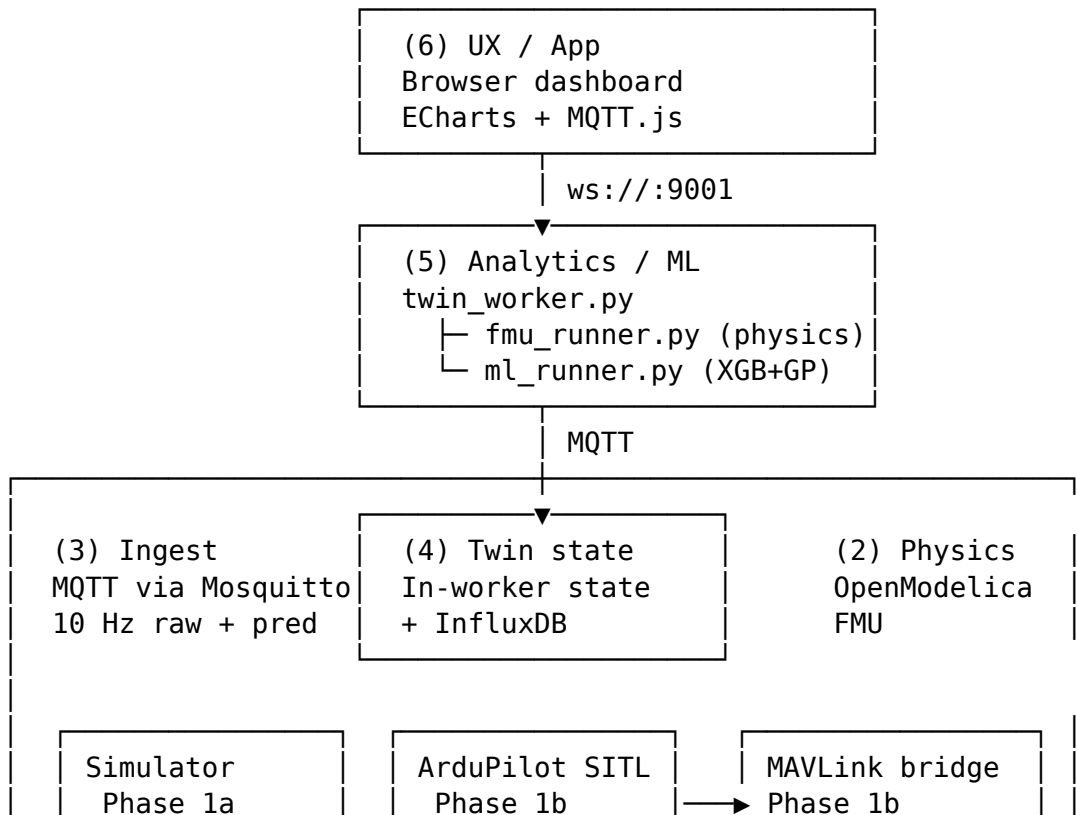
- **Phase 2 — Requirements gathering (done as Google Form)** — deployed as an operator survey with synthetic responses standing in for real interviews.
- **Phase 3a — ML data exploration + Model A training (done)** — notebooks 01/02/03/04 explored NASA IMS bearings, ALFA UAV faults, NASA PCoE batteries; trained XGBoost bearing classifier at CV AUC 0.9998.
- **Phase 3b — Model B Gaussian process (done)** — notebook 05 trained a Matérn 5/2 ARD GP on log-transformed RUL; 90% coverage 92.4%.
- **Phase 3c — Worker integration (done, verified 15 September 2026)** — ml_runner.py loads both models in the running worker; twin publishes fused physics + ML output on MQTT; three loose-thread fixes shipped (alert threshold, GP feature alignment, RPM synthesis).
- **Phase 4 — Real UAV** (next, once a design-partner operator is engaged).
- **Phase 5 — Ship as a product.**

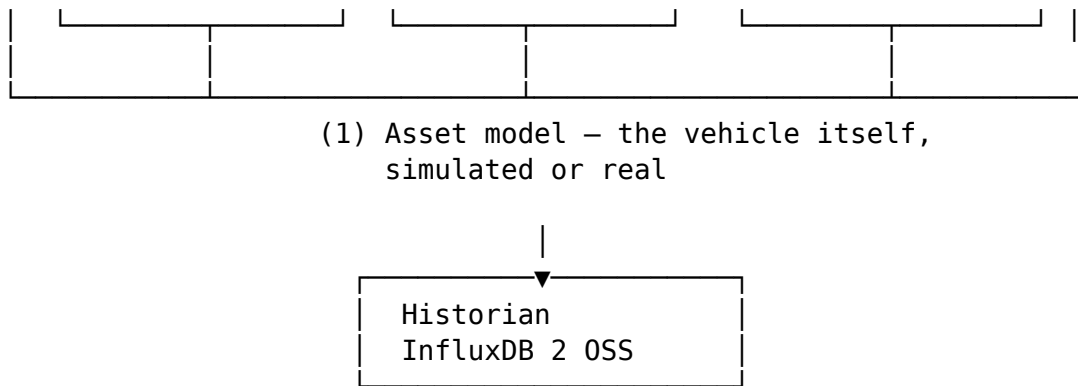
The strategic principle: **build one layer at a time, keep every interface stable, so no layer needs to be rewritten when a downstream layer is upgraded.** That is why the worker didn't change when 1a → 1b happened, didn't change when 1b → 1c happened, and only *added* code (didn't change existing) when 1c → 3c happened.

Part 3 — What we have built

3.1 The reference architecture

Every industrial-strength twin has six layers. We built all six with FOSS.





Layer numbers correspond to the classical six-layer twin taxonomy. Left-to-right, layer (1) — the asset — is currently the Python simulator or ArduPilot SITL; in Phase 4 it becomes a real UAV. Everything else stays the same.

3.2 Data flow — one telemetry sample from birth to death

Follow a single sample through the system at 10 Hz. It illustrates why the architecture is pluggable.

1. **Simulator / SITL** produces the current state of the (simulated) vehicle: motor RPMs, currents, temperatures, battery voltage, cell voltages, throttle command, flight mode. In Phase 1a this is Python code writing a JSON blob. In Phase 1b this is ArduCopter emitting MAVLink messages that the bridge translates into the same JSON blob.
2. The sample is published on MQTT topic `uav/quad-01/telemetry/state`.
3. **Mosquitto** delivers the sample to two subscribers: the twin worker (Python process) and the dashboard (browser).
4. **The twin worker** does three things now (Phase 3c):
 - Steps the OpenModelica FMU with the measured currents; reads back SoC, SoH, motor temps, motor damage. Publishes on `twin/state`.
 - Appends the sample to a 2-second rolling buffer; every 2 s, extracts 28 features and calls XGBoost. Publishes `p_fault` on `twin/anomaly`.
 - Every 1 Ah of pack throughput, snapshots battery state; every 30 s, calls the Gaussian process on the last 5 snapshots. Publishes RUL median + 90% CI on `twin/rul`.
 - If any threshold trips (motor asym > 3.5 A, temp spread > 8 °C, ML alert), publishes on `twin/alerts`.
 - Writes physics state as InfluxDB points every tick.
5. **The dashboard** receives all four twin topics over WebSocket MQTT (port 9001). It updates the rolling charts, the RUL tile with credible interval, the `p_fault` sparkline, and the alert list — all in the browser, no server-side rendering, no framework build step.
6. If the user clicks a fault-injection button, the dashboard publishes to `uav/quad-01/fault/inject`. The simulator subscribes to that topic and mutates its state (kills a motor, drops a cell). One tick later, the twin worker sees the effect through the same telemetry channel and starts emitting the corresponding alert.

Total end-to-end latency, measured on a laptop: roughly 30 ms from state generation to chart pixel. Well below human perception. The ML layer adds ~2 ms (XGBoost) + ~5 ms

(GP), still well inside the 100 ms per-tick budget.

3.3 Why MQTT

Every industrial twin ends up with a publish/subscribe message bus at its center. Direct API calls from every component to every other component do not scale past three services. MQTT is the standard for IoT because it is tiny (2-byte fixed header), reliable (QoS 0/1/2), broker-mediated (subscribers and publishers never need to know about each other), and has a WebSocket flavor so browsers can participate as first-class citizens. Every serious IoT platform on the market — AWS IoT Core, Azure IoT Hub, Google Cloud IoT — is fundamentally a managed MQTT broker with extras. Eclipse Mosquitto is the reference open-source implementation. The topic layout we use, `uav/<uav-id>/<what>/<subwhat>`, is the ISO/IEC 20922 recommended pattern.

3.4 The physics inside the twin worker (Phase 1c — OpenModelica FMU)

The worker holds a belief about the vehicle’s internal state and updates that belief every time a telemetry sample arrives. In Phase 1c the models are compiled Modelica — a declarative equation-based language whose OpenModelica compiler emits an FMI 2.0 co-simulation FMU. The FMU is loaded at worker startup via `fmpy` and stepped once per telemetry tick.

Model overview

`UAVPropulsion.mo` declares four blocks:

1. **Battery pack** — Coulomb-counted SoC and SoH, Thévenin equivalent circuit with SoC-dependent OCV and slightly SoC-dependent internal resistance. Cumulative Ah throughput drives a linear SoH decay.
2. **Per-motor electrical** — copper loss proportional to $I^2 R_{motor}$, plus a small quiescent draw.
3. **Per-motor thermal** — single-mass thermal model with convective cooling to ambient.
4. **Per-motor bearing damage** — Paris-law-inspired accumulator scaling with $I^{2.5}$ and an Arrhenius-lite temperature accelerator.

All state variables (SoC, SoH, `V_pack`, `V_ocv`, `Q_throughput`, `T_m0..T_m3`, `D_m0..D_m3`) are declared as output `Real` so they cross the FMI boundary to the Python worker.

Fallback path

If the FMU fails to load (file missing, wrong platform, corrupt binary), `fmu_runner.py` falls back to a pure-Python implementation of the same equations. The worker never goes dark. `[fmu_runner] FMU load failed ({e}); using Python fallback` is the log line; the twin’s mode field flips from `"fmu"` to `"python"` on every subsequent MQTT publish so the dashboard can display which physics path is active.

Formulas (still worth internalising)

Battery Thévenin:

$$V_{terminal}(t) = V_{OCV}(SoC(t)) - I_{pack}(t) \cdot R_{int}(SoC(t))$$

with $V_{OCV,cell}(SoC) = 3.3 + 0.9 \cdot SoC^{0.9}$ V/cell and $R_{int}(SoC) = R_0 + 0.005 \cdot (1 - SoC)$ Ω .

SoC Coulomb count:

$$SoC(t + \Delta t) = SoC(t) - \frac{I_{pack}(t) \cdot \Delta t}{3600 \cdot Q_{nom}(SoH(t))}$$

with $Q_{nom}(SoH) = 5.0 \cdot \max(SoH, 0.5)$ Ah.

SoH aging (linear-in-Ah for Phase 1c; Phase 3b replaced the *inference-time* forecast with a Gaussian process, but the FMU's per-tick integration remains this linear kernel — see §3.8):

$$SoH(t) = 1 - 10^{-4} \cdot Q_{throughput}(t)$$

Motor thermal:

$$C_{th} \cdot \frac{dT_{motor}}{dt} = I^2 R_{motor} + P_{idle} - h \cdot (T_{motor} - T_{ambient})$$

Bearing damage accumulator:

$$\dot{D}(t) = \dot{D}_{ref} \cdot \left(\frac{\max(0.1, |I(t)|)}{I_{ref}} \right)^{2.5} \cdot e^{0.06 \cdot (T(t) - T_{ref})}$$

with $I_{ref} = 8$ A, $T_{ref} = 45$ °C, bounded to $D \in [0, 1]$.

Fault injection through the FMU

The FMU takes only currents as inputs — the fault mechanism is upstream of it. When the user injects a motor-efficiency fault via the dashboard, the *simulator* (or the real vehicle later) responds with higher current to hold the commanded throttle. The FMU sees the elevated current, its thermal integrator climbs faster, and its damage accumulator ramps faster because of the $I^{2.5}$ exponent. This is the correct physics-of-failure ordering: fault modifies vehicle behavior, vehicle telemetry changes, twin's belief updates.

3.5 The MAVLink bridge — how Phase 1b works

What MAVLink is

MAVLink (Micro Air Vehicle Link) is a lightweight binary protocol for talking to autopilots. Every message is a fixed-format struct: header + payload + CRC, wire-encoded. Messages have IDs, and the protocol is versioned (we use MAVLink 2). ArduPilot and

PX4 both use it natively. ArduPilot extends the base MAVLink dialect with an ardupilotmega dialect that adds messages specific to ArduPilot (ESC_TELEMETRY_1_TO_4, various SIM_* and AP_* messages).

The key messages we consume in the bridge:

Message	Rate	What it carries
HEARTBEAT	1 Hz	Vehicle mode, armed/disarmed
SYS_STATUS	1 Hz	Pack voltage, pack current
BATTERY_STATUS	5 Hz	Per-cell voltages, current, temperature, remaining %
ESC_TELEMETRY_1_TO_4	10 Hz	Per-motor voltage, current, temperature (RPM synthesized — see below)
VFR_HUD	10 Hz	Throttle command, altitude, airspeed
ATTITUDE	10 Hz	Roll, pitch, yaw (not used yet — for Phase 4 mode-detection)

What the bridge does

It's a translation layer. It maintains a State object that accumulates the latest values from each MAVLink message type. At 10 Hz it snapshots that state into the JSON schema the Python simulator produced in Phase 1a, and publishes on the same MQTT topic. From the worker's point of view, nothing has changed — that's the whole point.

It also runs a background thread that uploads and starts a scripted mission (takeoff → four waypoints in a 40 m square → RTL land) on a loop, so the dashboard fills with continuous flight cycles instead of a single flight. MAVLink mission upload is a request/response dance: send MISSION_COUNT, receive MISSION_REQUEST_INT for each seq, send the MISSION_ITEM_INT for that seq, receive MISSION_ACK at the end.

RPM synthesis

SITL's ESC_TELEMETRY_1_TO_4 message reliably carries current, voltage, and temperature fields but leaves rpm at 0 — its motor model is a thrust-only abstraction with no rotational dynamics. The bridge synthesizes RPM whenever the reported value is zero and the vehicle is armed:

$$\text{rpm} = \text{throttle} \times V_{\text{pack}} \times \text{MOTOR_KV} \times \text{MOTOR_ETA}$$

with MOTOR_KV=900 and MOTOR_ETA=0.85 by default (env-overridable). At cruise (throttle 0.19, V_pack 16.8), this gives ~2440 RPM per motor — plausible for a 4S multicopter and enough signal for the XGBoost cross_min_rpm feature to carry information. When a real ESC arrives in Phase 4, the reported RPM > 0 and the synthesis path stops firing.

The TCP-vs-UDP fix

The Phase 1b initial code launched SITL with `sim_vehicle.py --no-mavproxy --out=udpout:mavlink-bridge:14550`. The bug: `--out` is a **MAVProxy** argument. With `--no-mavproxy`, MAVProxy never launches, and the `--out` argument is silently ignored — ArduCopter runs and streams MAVLink to nowhere. The bridge waits forever on an empty UDP port.

Fix: the ArduCopter binary itself listens on **TCP port 5760** as its primary MAVLink port. Connect the bridge to that instead. TCP is bidirectional so mission upload works over the same connection.

3.6 Docker and Docker Compose — the packaging story

Docker is a way to run software inside isolated containers. Each container has its own filesystem, its own network view, its own process tree — but they all share the host’s kernel (much lighter than a VM). A container is created from an **image**, which is a stack of read-only filesystem layers built from a Dockerfile.

Docker Compose is a way to describe a *set* of containers that run together, in one YAML file. Our `docker-compose.yml` defines seven services (mosquitto, influxdb, worker, dashboard, simulator, sitl, mavlink-bridge). Compose creates a private network between them; every service can reach every other service by its service name.

Compose profiles let one file express multiple deployment modes. We use two profiles, `sim` and `sitl`. Services without a profile are always on (the core stack). Services tagged with a profile only start when that profile is selected on the command line: `docker compose --profile sim up` starts core + simulator; `docker compose --profile sitl up` starts core + sitl + mavlink-bridge. This is how we ship two vehicle sources with one config file, and it’s how customers or research users could add their own profile for a different source without touching ours.

3.7 The licensing architecture

This is not decoration either. It is the reason the twin app you publish will never be blocked by a component’s license.

Our own code — Apache-2.0. Permissive, patent-grant included, compatible with almost every open component we use. If you eventually raise money or sell to a customer, Apache-2.0 does not scare anyone.

Every third-party component — kept at a process boundary, never linked into our binaries. OpenFOAM, MBDyn, ArduPilot, Grafana — the GPL and AGPL components — run in their own containers or as separate binaries. We talk to them over MQTT, over MAVLink sockets, over HTTP. Because our code never links to theirs at build time, we never inherit their copyleft obligations. This is the well-established “network use is not distribution” pattern.

Grafana skipped entirely. AGPL-3.0 requires that even users of a hosted service can request the modified source. That obligation is fine for a personal project but scares off future commercial customers. We built the dashboard as a first-party HTML page using

Apache-2.0 charting (ECharts) and MIT-licensed MQTT.js, and we vendored both locally in `dashboard/vendor/` so nothing is fetched from a CDN at demo time.

The upshot: when you publish this repo, another engineer can fork it, sell a hosted product built on it, and neither of you owes anything to any component vendor. That is what “no commercial-license blocker” actually looks like operationally.

3.8 Phase 3c — the ML layer is now live

The worker grew a sibling module `ml_runner.py` (mirrors `fmu_runner.py` in shape) that loads two pickled models at startup and calls them on every tick:

- **Model A** — XGBoost gradient-boosted trees over a 28-feature per-window vector, trained in notebook 04 on ALFA-style fault flights. CV AUC 0.9998 on synthetic. Threshold pinned at 0.5 operationally.
- **Model B** — Gaussian process regressor with a Matérn 5/2 ARD kernel on a 6-feature per-flight vector, trained in notebook 05 on NASA PCoE Li-ion aging. 90% coverage 92.4% on held-out cells. Output is a log-Gaussian back-transformed to (median, p05, p95) flights remaining.

Both models save a `.pkl` (weights) + `.json` (contract: feature order, threshold, provenance, CV metrics) pair. The worker verifies the feature order at inference time and returns a safe fallback (`p_fault=0.0` or `unavailable: true`) if any component of the ML layer misbehaves. The rest of the twin — physics, historian, dashboard — is unaffected.

Four MQTT topics carry the fused output:

- `uav/quad-01/twin/state` — every tick, full FMU state (SoC, SoH, temps, damage) plus a physics field indicating `fmu` vs `python-fallback`.
- `uav/quad-01/twin/anomaly` — every 2 s, XGBoost `p_fault`, cross-motor asymmetry, mode field.
- `uav/quad-01/twin/rul` — every 30 s, GP median + p05 + p95, or `unavailable:true`, `reason:"warming_up"` for the first ~14 min of continuous flight.
- `uav/quad-01/twin/alerts` — discrete events (motor asym > 3.5 A, cell spread > 8 °C, ML fault > 0.5).

For the operational reference — every threshold, every env var, every payload schema, every known limitation — see the companion doc `phase-3c-complete.md`.

Part 4 — What comes next

4.1 Phase 3c retrospective (September 2026)

Three loose threads emerged during Phase 3c integration and were closed in the same session:

- **Alert threshold calibration** — Model A’s notebook-derived `operational_threshold` (0.000673) was optimal on training data but read as “always alert” on real telemetry. Pinned at 0.5 operationally with the notebook value preserved for reproducibility.
- **GP feature schema alignment** — the worker’s `_FlightHistory.features()` originally returned a superset of keys that didn’t match `battery_rul.json:feature_order`.

`ml_runner.battery_rul()` silently returned `unavailable: true`. Fixed to return exactly the 6 keys the model expects.

- **Motor RPM synthesis** — SITL's ESC_TELEMETRY message left rpm at 0. Bridge now synthesizes RPM from $\text{throttle} \times \bar{V} \times \text{KV} \times \eta$ when the reported value is 0 and the vehicle is armed.

Two encoding bugs are also worth naming, because they will recur if not guarded:

- Windows-1252 (CP1252) encoded Python source with em-dashes in comments fails to parse under Python's UTF-8 default. `twin_worker.py` and `ml_runner.py` are pinned pure-ASCII as a belt-and-braces guard.
- Windows PowerShell 5.1's `Get-Content -Encoding` does not accept Windows-1252 as a name; use `[System.Text.Encoding]::GetEncoding(1252)` via .NET directly.

4.2 What comes after Phase 3c, ordered by cost-to-value

1. **Dashboard update (2-3 hours)** — a browser tile row that renders the four twin topics: SoC/SoH gauges, motor-temp bar row, motor-damage sparklines, `p_fault` needle with alert-threshold overlay, RUL card with the 90% CI, alert log. Turns the terminal-only twin into a demoable web page. The infrastructure (MQTT.js, ECharts, vendored assets) is already in `dashboard/`.
2. **Public GitHub push (one evening)** — clean the repo, write a proper README, LICENSE headers, one-command up. Portfolio artifact.
3. **RPM synthesis validation with real ESC data** — once you have a real UAV and can compare synthesized vs measured RPM, tune `MOTOR_KV` and `MOTOR_ETA` from the discrepancy. Same synthesis path stays useful as a fallback when ESC telemetry drops.
4. **Real fleet retrain (Phase 4)** — swap the notebook synthetic loaders for real ArduPilot/PX4 logs from a design-partner operator. Re-run notebooks 04 and 05; `.pkl` files regenerate; worker picks them up on next rebuild. Nothing else changes.
5. **Model C — unsupervised anomaly detector (notebook 07)** — isolation forest or one-class SVM on the same 28-feature space, to catch failure modes neither the FMU nor XGBoost recognises. Best deferred until real fleet data exists.
6. **Continuous re-training pipeline** — every 30 days, retrain on the last 90 days of fleet telemetry, A/B against the incumbent, only ship if AUC improves and FPR doesn't grow. Standard MLOps practice.

4.3 Phase 4 — real UAV

The bill of materials for a **research quadrotor** — you own it, log everything:

Item	Rough cost	Purpose
Holybro X500 v2 kit (or equivalent 500-mm carbon frame + 3520/920 KV motors + BLHeli32 ESCs + Pixhawk 6C)	₹45k	The airframe; ArduPilot / PX4-compatible
4S 5000 mAh Li-ion pack (Tattu R-Line or equivalent)	₹4k	Battery under test

Item	Rough cost	Purpose
Current-shunt logging (bench) — Otii Arc or CorePower	₹35k	Ground-truth current telemetry richer than the ESC's
Herelink or SiK radio for telemetry link	₹15k	MAVLink to ground during flight
Miscellaneous — spares, props, chargers, tools	₹15k	The little things that eat weekends

Total ~₹1.1-1.5 lakh. Do not buy until Phase 2 conversations have produced at least one operator who would be a design partner — buying first is a common mistake.

Flight test sequence: 1. **Bench** — motors on a thrust stand, battery on a discharge bench, get calibrated per-component curves. This is the golden dataset for tuning the Modelica FMUs. 2. **Tethered hover** — fly with a physical tether inside a net enclosure (many Indian universities have one). Log everything to onboard SD + real-time MAVLink to your MQTT bridge. The twin is now consuming *real* telemetry. 3. **Free flight** on a certified airstrip / DGCA-approved test area. Inject controlled faults in a safe environment — mild ESC current-limit reductions, controlled cell imbalance — and verify the twin catches them.

4.4 Phase 5 — ship as a product

By this phase we know what to build and who buys it. Choices to sequence:

Business model — three viable shapes: **SaaS** (they upload flight logs to your cloud, you run the twin, monthly per-vehicle fee); **on-premise appliance** (a small server they install in their hangar, receives telemetry over the local network); **open-core** (the twin is free, paid features are enterprise auth / multi-tenant / integration with their scheduling software). Pick after Phase 2 tells you what they want.

Regulatory posture — for pure post-flight analytics on a customer's own data, there is essentially no aviation regulation to satisfy in India. If we ever cross into *in-flight* recommendations that the pilot acts on, we're in DGCA territory. Keep this line clean until Phase 4 and the answer is easy: post-flight only for the first 18 months.

Distribution — a plain web app is fine. For MRO shops we build a small “kiosk mode” for the tablet next to the maintenance bench. Mobile app is a distant Phase 5.5.

Part 5 — Reference

5.1 File layout

```

uav-twin/
├── docker-compose.yml           # profiles: sim (1a), sitl (1b)
├── mosquito/config/
│   └── mosquito.conf
├── worker/
└── Dockerfile                   # two-stage: OpenModelica → python:3.12-slim

```

— requirements.txt	# paho-mqtt, numpy, influxdb-client, fmpy, # xgboost, scikit-learn
— twin_worker.py	# physics + ML + alerts + Influx (pure ASCII)
— fmu_runner.py	# FMU wrapper + Python fallback
— ml_runner.py	# XGBoost + GP loader (pure ASCII)
— models/	
— UAVPropulsion.mo	# Modelica source
— UAVPropulsion.fmu	# compiled FMU (co-simulation, FMI 2.0)
— bearing_wear.pkl	# XGBoost
— bearing_wear.json	# feature order + threshold + CV metrics
— battery_rul.pkl	# GP + StandardScaler bundle
— battery_rul.json	# feature order + kernel + CV metrics
— simulator/	# Phase 1a Python quadrotor (sim profile)
— sitl/	
— Dockerfile	# ArduCopter build from source
— defaults.parm	# BATT_MONITOR=4, SIM_ESC_TELEM=1, # mAh-failsafes disabled
— start-sitl.sh	
— mavlink-bridge/	
— Dockerfile	
— bridge.py	# MAVLink -> MQTT + RPM synthesis
— mission.py	# scripted mission uploader + force-arm
— requirements.txt	
— dashboard/	
— index.html	# single-page dashboard
— vendor/	# ECharts + MQTT.js, vendored
— notebooks/	# Phase 3a notebooks + utils + data
— docs/	# architectural notes
— LICENSE	# Apache-2.0
— README.md	

5.2 Command cheatsheet

Phase 1a – fast Python simulator

`docker compose --profile sim up --build`

*# Phase 1b + 1c + 3c – ArduPilot SITL with FMU physics + ML layer (first build
↪ ~15 min)*

`docker compose --profile sitl up --build`

Watch a service's logs

`docker compose --profile sitl logs -f worker`

Rebuild just one service after editing its source

`docker compose --profile sitl up -d --build worker`

Peek at MQTT traffic

`docker exec uavtwin-mosquitto mosquitto_sub -h localhost -t 'uav/#' -v -C 12`

```

# Peek at just twin outputs
docker exec uavtwin-mosquitto mosquitto_sub -h localhost -t 'uav/+/twin/#' -v
↪ -C 12

# Tear down (keeps volumes)
docker compose --profile sitl down

# Tear down and wipe stored data
docker compose --profile sitl down -v

# List running containers
docker ps

# Free up disk from unused images
docker system prune -a

```

5.3 Topic map

MQTT topic (under uav/<uav-id>/)	Publisher	Rate	Payload
telemetry/state	sim OR bridge	10 Hz	Full vehicle state snapshot
twin/state	worker	10 Hz	FMU state: SoC, SoH, V_pack, motor temps, motor damage
twin/anomaly	worker	0.5 Hz	XGBoost p_fault, alert bool, cross-motor features
twin/rul	worker	0.033 Hz	GP RUL median + p05 + p95 flights remaining, warmup state
twin/alerts	worker	on trigger	Discrete alerts (motor asym, temp spread, ML fault)
fault/inject	dashboard	on click	Fault-injection command

5.4 Glossary

BLDC — Brushless DC motor. What every drone uses. **ESC** — Electronic Speed Controller. The power-electronics box between battery and motor. **FMI / FMU** — Functional Mock-up Interface / Unit. Open standard + file format for portable simulation models. **GP** — Gaussian process. A nonparametric Bayesian regressor with calibrated uncertainty. **HIL / SITL** — Hardware / Software In The Loop simulation modes. **KV** — Motor no-load RPM per volt. A 900 KV motor spins at $900 \times V_{\text{pack}}$ RPM at no load.

MAVLink — Micro Air Vehicle Link. Binary protocol for talking to autopilots. **MCSA** — Motor-Current Signature Analysis. Using motor current spectrum to detect mechanical faults. **MQTT** — Message Queuing Telemetry Transport. Lightweight pub/sub messaging protocol used everywhere in IoT. **MRO** — Maintenance, Repair, and Overhaul. The industry we're selling to. **OEM** — Original Equipment Manufacturer. **ROM** — Reduced-Order Model. A fast approximation of a slow high-fidelity model. **RUL** — Remaining Useful Life. Predicted time-to-failure of a component. **SoC / SoH** — State of Charge (how full) / State of Health (how degraded). **XGBoost** — Extreme Gradient Boosting. Tree-ensemble supervised learner, the default first-pick for tabular ML.

5.5 Where things came from

Prior project docs (in your Claude project): - `claude/phase-0-kickoff.md` — landscape and initial scope reasoning - `claude/phase-0-decisions.md` — locked-in scope, tooling, six-decision resolution, glossary - `claude/phase-1a-readme.md` — the Phase 1a README - `claude/phase-2-interview-kit.md` — the interview kit (before pivoting to Google Form) - `claude/phase-2-google-form-kit.md` — the operator survey - `claude/phase-3-kickoff.md` — the ML-layer scoping doc - `claude/phase-3a-study-guide.md` — the notebooks 01-06 walkthrough - `claude/phase-3c-complete.md` — the operational reference for Phase 3c (companion to this doc)

External references worth reading in order of usefulness: - [Rolls-Royce IntelligentEngine](#) — the reference for a mature aviation twin - [The Digital Twin Paradigm for Aircraft \(AIAA\)](#) — canonical review - [ISO 23247 \(NIST analysis\)](#) — the standards framework - [ALFA UAV fault dataset](#) - [NASA PCoE Data Repository](#) - [PX4 SITL simulation guide](#) — better docs than ArduPilot's for the same underlying idea - [OpenModelica battery library docs](#) - [Modelica / FMI beginners tutorial](#) — read before Phase 1c